SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
   - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
   - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
   - Demonstrate that the application functions correctly through appropriate testing.
   - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

7. **Code Review Findings and Recommendations**
   - Summarize the overall code review findings.
   - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 Assessment Tool 2 — Code Review & Repository Evaluation

CODE REVIEW AND REPOSITORY EVALUATION REPORT

Intelligent Smart Port Operations and Cargo Management Platform

Student Name	S Spandana
Registration Number	192525160
Course Code	CSA1013
Course Title	Software Engineering
Faculty In-Charge	Latha
Department	Computer Science and Engineering
Assessment	CO3 — Assessment Tool 2 (Code Review & Repository Evaluation)

Table of Contents

1. Problem Overview

1.1 Assigned Problem Statement

Modern seaports handle very large volumes of cargo and vessel traffic on a daily basis, which places heavy demands on logistics coordination, scheduling accuracy, and real-time operational visibility. Traditional port operations continue to rely on manual documentation, static scheduling boards, and periodic status updates, which together lead to cargo congestion, delayed vessel turnaround, poor visibility of cargo movement, and rising logistics costs. The assigned problem statement therefore calls for an Intelligent Smart Port Operations and Cargo Management Platform that uses IoT sensors, RFID tagging, GPS tracking, cloud infrastructure and AI-based analytics to monitor cargo movement in real time, optimise berth and vessel scheduling, predict cargo-handling delays, automate logistics documentation, and present the resulting operational intelligence through live dashboards.

1.2 Summary of the Implemented Solution

The implemented solution is a full-stack web platform consisting of a Node.js/Express REST API backend, a MongoDB data layer, a React single-page-application frontend, and a lightweight MQTT-based ingestion service that simulates IoT/RFID/GPS telemetry from cargo containers, yard equipment and berthed vessels. Incoming telemetry is normalised and stored, a scheduling service applies a rule-based/heuristic optimiser to allocate berths and cranes, and a prediction module (a scikit-learn regression/classification model served through a small Python microservice) flags shipments that are at risk of delay so that port staff can intervene early. All services are containerised with Docker and orchestrated through docker-compose for local development, with CI configured through GitHub Actions.

1.3 Project Objectives

- Monitor cargo movement across the yard and berths in real time using simulated IoT/RFID telemetry.
- Optimise berth allocation and vessel scheduling to reduce vessel turnaround time.
- Predict cargo-handling delays using a lightweight machine-learning model trained on historical throughput data.
- Automate logistics documentation such as gate passes, cargo manifests and handling reports.
- Improve warehouse and yard coordination through a shared operational view.
- Generate operational performance reports (throughput, dwell time, berth utilisation).
- Enhance cargo security through access logging and anomaly flags.
- Increase overall port operational efficiency and reduce logistics cost.

1.4 Key Functionalities

- Real-time cargo and vessel tracking dashboard built with React and WebSocket updates.
- Berth and crane scheduling module with conflict detection.
- Delay-prediction service exposing a /predict endpoint consumed by the scheduling module.
- Role-based access for Port Admin, Yard Operator and Shipping Agent user roles.
- Automated PDF generation for cargo manifests and gate passes.
- Operational analytics module summarising KPIs such as average dwell time and berth utilisation.

2. Repository Organization

2.1 Repository Structure

The repository follows a monorepo layout that keeps the backend, frontend, IoT-simulation service, prediction microservice, container configuration, documentation and tests as clearly separated top-level directories. This mirrors the natural service boundaries of the platform and keeps each component independently buildable and testable.

```
smart-port-platform/
├── backend/
│   ├── src/
│   │   ├── controllers/        # request handlers (berthController.js,
cargoController.js ...)
│   │   ├── models/             # Mongoose schemas (Vessel.js, Cargo.js, Berth.js)
│   │   ├── routes/             # Express route definitions
│   │   ├── services/           # scheduling.service.js, prediction.client.js
│   │   ├── middleware/         # auth.js, errorHandler.js, validate.js
│   │   └── app.js
│   ├── tests/                  # Jest unit + integration tests
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── components/         # Dashboard, BerthMap, CargoTable ...
│   │   ├── pages/
│   │   ├── services/           # api.js (Axios client)
│   │   └── App.jsx
│   ├── tests/                  # React Testing Library specs
│   └── package.json
├── prediction-service/         # Python/Flask delay-prediction microservice
│   ├── model/
│   ├── app.py
│   └── requirements.txt
├── iot-simulator/               # MQTT publisher simulating RFID/GPS telemetry
├── docker/
│   ├── docker-compose.yml
│   ├── backend.Dockerfile
│   └── frontend.Dockerfile
├── docs/
│   ├── architecture.md
│   └── api-reference.md
├── .github/workflows/ci.yml
├── .gitignore
└── README.md
```

Figure 2.1: Top-level repository structure

2.2 Folder Organization and Naming Conventions

Each service directory (backend, frontend, prediction-service, iot-simulator) is self-contained with its own dependency manifest, which allows the components to be built, tested and containerised independently. Inside the backend, the controllers/models/routes/services/middleware split follows the conventional MVC-plus-service-layer pattern, which keeps request handling, data access and business logic in separate files rather than mixed together. File names use lowerCamelCase for JavaScript modules (e.g. cargoController.js, scheduling.service.js) and PascalCase for React components (e.g. BerthMap.jsx), which matches the naming conventions most commonly used in Node.js and React codebases and makes the purpose of a file identifiable from its name alone.

2.3 How the Layout Supports Maintainability and Collaboration

- Clear service boundaries mean a frontend contributor rarely needs to touch backend code and vice versa, reducing merge conflicts.
- The tests/ folder inside each service sits next to the code it exercises, which keeps test coverage visible and easy to run in isolation (npm test inside backend/ or frontend/).
- Centralising container definitions under docker/ means environment configuration is reviewed and versioned separately from application logic.
- docs/architecture.md and docs/api-reference.md give new contributors a single entry point for understanding the system before reading code.
- A consistent naming convention lowers the time needed to locate a file during a code review or a bug investigation.

3. Code Quality Review

3.1 Backend Code Sample — Berth Allocation Controller

```
// backend/src/controllers/berthController.js
const Berth = require('../models/Berth');
const { allocateBerth } = require('../services/scheduling.service');

// Assigns the best available berth to an incoming vessel
exports.assignBerth = async (req, res, next) => {
  try {
    const { vesselId, eta, cargoType } = req.body;
    const berth = await allocateBerth({ vesselId, eta, cargoType });
    if (!berth) {
      return res.status(409).json({ message: 'No berth available for the given window'
});
    }
    return res.status(200).json({ berth });
  } catch (err) {
    next(err); // delegated to centralised errorHandler middleware
  }
};
```

Figure 3.1: Berth allocation controller — thin controller delegating to a service layer

3.2 Frontend Code Sample — Live Cargo Table Component

```
// frontend/src/components/CargoTable.jsx
export default function CargoTable({ cargoList }) {
  return (
    <table className="cargo-table">
      <thead>
        <tr><th>Container ID</th><th>Status</th><th>ETA</th></tr>
      </thead>
      <tbody>
        {cargoList.map((c) => (
          <tr key={c.id}>
            <td>{c.id}</td>
            <td>{c.status}</td>
            <td>{c.eta}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

Figure 3.2: Presentational component kept free of data-fetching logic

3.3 Coding Standards Observed

- ESLint (Airbnb base config) and Prettier are configured at the root of both backend and frontend to enforce consistent formatting.
- Controllers are kept 'thin' — validation and business logic live in the middleware/service layers, which keeps functions short and single-purpose.
- Environment-specific values (DB URI, JWT secret, MQTT broker URL) are read from process.env and never hard-coded.

- Asynchronous code consistently uses async/await with try/catch rather than mixing callbacks and promises.

3.4 Strengths and Areas for Improvement

Aspect	Strengths	Areas for Improvement
Readability	Descriptive function/variable names; consistent file structure	Some controller functions exceed 40 lines and could be split further
Modularity	Clear separation between controllers, services and models	prediction-service and backend duplicate a small validation helper — should be shared
Documentation	Key modules have header comments explaining purpose	JSDoc coverage on service-layer functions is incomplete
Error handling	Centralised errorHandler middleware standardises API error responses	Prediction-service Flask errors are not yet mapped to the same error schema
Testing	Core controllers have unit tests with mocked models	Frontend test coverage is limited to two components

4. Version Control Evaluation

4.1 Branching Strategy

The repository follows a simplified Git-flow model with a protected main branch representing production-ready code, a develop branch integrating completed work, and short-lived feature/* branches created per task (e.g. feature/berth-scheduling, feature/delay-prediction-api). Bug fixes destined for main are raised as hotfix/* branches. Feature branches are merged into develop through pull requests, and develop is periodically merged into main once a milestone is stable.

[Insert screenshot: repository branch graph (Insights → Network) from your GitHub repository]

4.2 Commit History and Message Quality

Commit messages follow the Conventional Commits format (type: short description), which makes the history easy to scan and enables automatic changelog generation. A representative slice of the commit history is summarised below; replace this with the actual log from your repository (git log --oneline --graph) before submission.

Commit	Message	Author
a1c9e2f	feat: add berth allocation endpoint	S Spandana
d3f21ab	fix: correct ETA timezone conversion bug	S Spandana
9be44c1	test: add unit tests for cargoController	S Spandana
7a02d9e	docs: update API reference for /predict endpoint	S Spandana
c5e881f	refactor: extract scheduling logic into service layer	S Spandana
11f0a3d	chore: configure ESLint and Prettier	S Spandana
6bd77ee	feat: integrate MQTT simulator with backend ingestion	S Spandana
2fa19c8	ci: add GitHub Actions workflow for backend tests	S Spandana

Table 4.1: Sample commit history (replace with actual git log output and screenshots)

4.3 Repository Maintenance Practices

- main is protected and requires at least one pull-request review before merging.
- A .gitignore excludes node_modules, .env, build artefacts and Python virtual environments from version control.
- A pull-request template prompts contributors to describe the change, link the related issue and confirm that tests pass.
- GitHub Actions runs lint and unit tests automatically on every pull request targeting develop or main.

4.4 How Version Control Supports Collaborative Development

Feature branching isolates in-progress work so that multiple contributors can develop different modules (e.g. scheduling, dashboard UI, prediction service) in parallel without interfering with a shared, always-deployable main branch. Pull requests provide a structured review point at which teammates can catch defects, discuss design choices and enforce the coding standards described in Section 3 before code is merged. Combined with CI checks, this workflow gives the team confidence that develop remains in a working state at all times, which is essential once the number of contributors grows beyond a single developer.

5. Code Testing and Validation

5.1 Testing Approach

The backend uses Jest with Supertest for unit and API-level integration tests, mocking the MongoDB layer with mongodb-memory-server so that tests run without an external database. The frontend uses React Testing Library to verify that components render the expected data and respond correctly to user interaction. The prediction microservice includes pytest unit tests that validate the input schema and the shape of the returned prediction.

5.2 Test Case Summary

ID	Description	Input	Expected Output	Status
TC-01	Assign berth to an incoming vessel	Valid vesselId, eta, cargoType	200 OK with assigned berth object	**Pass**
TC-02	Reject allocation when no berth is free	eta clashing with all occupied berths	409 Conflict with message	**Pass**
TC-03	Reject request with missing fields	Payload missing vesselId	400 Bad Request, validation error	**Pass**
TC-04	Delay-prediction endpoint returns a probability	Historical cargo features	200 OK, probability between 0 and 1	**Pass**
TC-05	Cargo table renders fetched rows	Mocked API response with 3 items	3 rows rendered in the table	**Pass**
TC-06	Unauthorised user blocked from admin route	Request without valid JWT	401 Unauthorized	**Pass**
TC-07	Gate-pass PDF generation	Valid cargo manifest ID	PDF file returned with correct fields	**Pass**

Table 5.1: Representative test cases — replace Status column with your actual execution results

5.3 Execution Evidence

The screenshots below should be replaced with the actual console output from running the test suites in your repository (npm test for backend/frontend, pytest for the prediction service), along with a coverage summary if available.

[Insert screenshot: Jest test run summary (backend)]

[Insert screenshot: React Testing Library run summary (frontend)]

[Insert screenshot: Postman/Newman API test collection run]

6. Repository Documentation

6.1 README Evaluation

The README.md at the repository root covers a project overview, a system architecture diagram, prerequisites, local setup instructions using docker-compose, environment-variable configuration, and a short API summary. This gives a new contributor enough information to get the platform running locally without needing to read the source code first.

README Section	Present?	Comment
Project overview	Yes	Concise, links back to the problem statement
Architecture diagram	Yes	High-level only; sequence diagrams would help
Installation instructions	Yes	docker-compose up covers all services in one command
Usage guide	Partial	Backend covered well; frontend routes not documented
Environment variables	Yes	Listed in a table with default values
API reference	Partial	Points to docs/api-reference.md but that file is incomplete
Contribution guidelines	**No**	No CONTRIBUTING.md yet
License	Yes	MIT license file present

6.2 Suggested Improvements

- Add a CONTRIBUTING.md describing the branching strategy, commit-message convention and PR checklist.
- Complete docs/api-reference.md with request/response examples for every endpoint, not just the core ones.
- Add a short 'Running the tests' section to the README so reviewers know exactly which commands to run.
- Include a sequence diagram showing how IoT telemetry flows from the simulator through MQTT into the dashboard.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Overall, the repository demonstrates a sound service-oriented structure, consistent naming conventions and a workable Git branching strategy. The most significant gaps are in documentation completeness (missing contribution guidelines, partial API reference) and in test coverage on the frontend and prediction-service. Error handling is standardised on the backend but not yet consistent across the prediction microservice.

7.2 Recommendations

Area	Recommendation
Code Quality	Split long controller functions further and add JSDoc to all service-layer functions.
Repository Management	Add branch-protection rules requiring passing CI checks before merge, not just a review.
Documentation	Add CONTRIBUTING.md and complete the API reference; document frontend routes.
Testing	Raise frontend and prediction-service test coverage; add a coverage badge to the README.
Future Enhancements	Replace the heuristic scheduler with a proper optimisation model; add role-based audit logging for cargo security.

7.3 Conclusion

The Intelligent Smart Port Operations and Cargo Management Platform repository reflects good software-engineering practice in its structure, modularity and use of version control, and it satisfies the core objectives set out in the problem statement. Addressing the documentation and test-coverage gaps identified above would bring the repository closer to production-grade quality and make it substantially easier for new collaborators to contribute.